

VocalNote Project Brief

A presentation-ready summary of the project, architecture, workflow, stack, and talking points

Prepared from the current repository snapshot on 2026-06-27

One-line pitch: VocalNote turns a PDF into a natural two-speaker podcast entirely on the user's machine, using local language models and local text-to-speech instead of cloud APIs.

Purpose of this document: give a teammate everything needed to build a clean PPT for a project presentation without reading the full codebase.

Project Snapshot	Details
Project name	VocalNote
Core idea	Convert PDF documents into a two-speaker podcast conversation
Primary input	A user-uploaded PDF file
Primary output	A downloadable WAV podcast file
Privacy model	Runs locally; no cloud API calls are required after setup
Target users	Students, researchers, professors, and professionals who prefer audio learning
Frontend	React + TypeScript + Vite on port 2000
Backend	FastAPI on port 9000
AI components	Qwen3 1.7B for preprocessing, Qwen2.5 3B for transcript generation, Kokoro-82M for TTS

1. Executive Summary

VocalNote is a local AI application that transforms a static PDF into a conversational podcast. The system accepts a PDF, extracts and cleans its text, rewrites the content into a two-speaker discussion, and synthesizes that script into audio. The project combines usability, privacy, and local inference performance into a single end-to-end product.

- It solves a real learning problem: long PDFs are hard to consume, especially when users want to study while walking, commuting, or multitasking.
- It emphasizes privacy and offline use, which is valuable for sensitive academic, legal, or internal business documents.
- It is a full-stack AI product rather than just a model demo: backend pipeline, frontend UI, deployment story, and user download flow are all included.

Presentation angle: this is strongest when positioned as a practical edge-AI product that makes reading more accessible and flexible.

2. Problem Statement And Motivation

- Many important documents are long, dense, and tiring to read line by line.
- Traditional document-to-audio tools are often cloud based, require API keys, or raise privacy concerns.
- Users may want audio learning without uploading private files to external services.

- The team wanted to show that modern AI pipelines can run on consumer hardware with good UX.

Core motivation: make document consumption more engaging, more private, and more convenient through local AI.

3. Solution Overview

The project provides a simple user flow. A user uploads a PDF in the web app, clicks generate, waits while the backend processes the document, then previews and downloads the finished WAV file.

Stage	What happens
1. Upload	Frontend validates that the selected file is a PDF and sends it to the backend as multipart form data.
2. Extraction	Backend reads the PDF and extracts text with PyPDF2, limiting extraction to roughly 100,000 characters.
3. Cleaning	A local Qwen model removes noise, duplicates, formatting artifacts, links, and low-value text.
4. Script generation	A second local Qwen model rewrites the cleaned content into a two-speaker podcast transcript.
5. Audio generation	Kokoro TTS generates speaker audio chunks and concatenates them into one WAV file.
6. Delivery	Frontend receives the audio blob, creates a browser audio URL, and lets the user play or download it.

4. System Architecture

Layer	Responsibility	Key files
Frontend	Upload UI, status display, audio playback, download flow	frontend/src/App.tsx and component files
API layer	Receives uploads, triggers the pipeline, returns generated WAV	backend/api.py
Pipeline layer	PDF extraction, cleaning, transcript generation, TTS synthesis	backend/podcast_workflow.py
Models and runtime	Local GGUF LLMs and Kokoro voice synthesis	models directory at runtime
Deployment	Docker compose for split services and a monolithic Dockerfile for bundled delivery	docker-compose.yml and Dockerfile

Architecture summary: the product is intentionally simple from the user side, but internally it is a multi-stage AI pipeline wrapped in a full-stack application.

Useful presenter note: this separation makes it easy to explain the project in layers - UI, API, AI pipeline, then deployment.

5. Detailed Pipeline Walkthrough

The current backend pipeline is implemented in a class named PodcastPipeline. It uses temporary files for each stage so the process is easy to inspect and debug.

- Temporary working files include the uploaded PDF, extracted text, cleaned text, transcript pickles, and the final podcast WAV.
- PDF extraction checks the file type, reads all pages, and stops once the maximum character budget is reached.
- Text cleaning uses chunking of about 3,000 characters to keep local inference manageable.
- The transcript generator uses a larger context window and instructs the model to produce a Python list of speaker tuples.

- The TTS step assigns one voice to Speaker 1 and another to Speaker 2, then concatenates all segments in memory.

Pipeline detail	Code-backed behavior
Input validation	Only PDF uploads are accepted.
Character cap	Extraction is capped around 100,000 characters to control runtime.
Transcript format	The model is asked to return a Python list of tuples like ("Speaker 1", "...").
Voices	Speaker 1 uses am_fenrir and Speaker 2 uses bf_emma in Kokoro.
Output format	The generated file is written as podcast.wav and returned to the client.

6. AI And Performance Optimizations

One of the strongest parts of the project is that it is not just using local models - it is also optimizing how those models are used.

- Hardware detection: the app checks whether it is running on Apple Silicon and can enable Metal/MPS acceleration outside Docker.
- CPU fallback: on non-macOS systems or when GPU acceleration is unavailable, it falls back to CPU execution with controlled threading.
- Lazy loading and caching: model instances are loaded once and reused, which avoids repeated cold-start overhead.
- Merged workflow: transcript generation and rewrite are effectively collapsed into a single structured generation pass.
- In-memory audio concatenation: NumPy arrays are combined directly before writing the final WAV, which avoids slow repeated file append operations.
- Model auto-download: if model files are missing, the backend can fetch them automatically on first run.

Performance message for the PPT: the project is designed not only to work locally, but to work locally in a realistic and user-friendly way.

7. Frontend And User Experience

The frontend is more than a plain upload form. It tries to make the product feel premium and culturally distinct.

- The app uses React with a central App component that manages upload state, processing state, errors, and audio playback state.
- The upload flow validates file type on the client before sending the request.
- A status bar communicates progress, success, or failure and warns users that generation may take several minutes.
- A custom audio player allows play, pause, seek, and download of the generated WAV file.
- The design language uses Indian-inspired visual references such as Jaipur tiles, mandala motifs, Mughal arch card shapes, and diya-inspired status animation.

UI element	Purpose
Hero upload section	Primary interaction area for selecting the PDF and starting generation
Status bar	Shows processing, success, and error states
Audio player	Lets users preview and seek through the generated podcast

UI element	Purpose
Download button	Downloads the final WAV file directly
Feature cards	Explain the product as a three-step story: PDF extraction, local AI models, natural voices

8. Tech Stack

Category	Technologies used	Why they matter
Frontend	React, TypeScript, Vite, Axios	Fast UI development, typed state handling, and clean API calls
Backend	Python 3.11, FastAPI, Uvicorn	Async-friendly API service and simple deployment
PDF processing	PyPDF2	Extracts text from uploaded documents
Text preprocessing	LangChain text splitters, regex cleanup, local Qwen model	Prepares noisy document text for script generation
Language models	llama-cpp-python with GGUF models	Local inference without cloud APIs
TTS	Kokoro-82M	Generates natural speech for two speakers
Audio processing	NumPy, SciPy, Pydub	Audio buffering, concatenation, and WAV export
Deployment	Docker, Docker Compose, multi-stage builds	Simplifies setup and sharing across machines

9. API Endpoints

Endpoint	Method	Purpose
/upload-pdf	POST	Uploads and stores a PDF before generation.
/generate-podcast	POST	Runs the full pipeline and returns a WAV file.
/download-podcast	GET	Returns the last generated podcast file if present.
/	GET	Serves the built frontend if static assets exist in dist.

Important integration detail: the frontend currently posts directly to `http://localhost:9000/generate-podcast` and expects a blob response.

10. Deployment Story

The project has multiple ways to run, which is useful in a presentation because it shows the team thought about developer setup and end-user setup separately.

- Traditional local setup: run the backend and frontend separately for development.
- Docker Compose setup: run backend and frontend as two services for easier onboarding.
- Monolithic Docker image: build the frontend, copy dist into the Python image, and serve the app from FastAPI.
- Model persistence: Docker mounts the models directory to the user host machine so huge files are not repeatedly re-downloaded.
- Cross-platform consideration: Apple Silicon GPU acceleration works in native local runs, but Docker on macOS falls back to CPU.

11. Key Strengths

- End-to-end product thinking: upload, process, preview, and download are all included.
- Clear privacy value proposition because documents stay on the local machine.
- Strong use of local AI rather than dependency on external APIs.
- Presentation-friendly concept: easy for judges or classmates to understand quickly.
- Technical depth across NLP, speech synthesis, web development, and deployment.
- Distinctive visual identity in the frontend instead of a generic AI demo interface.

12. Limitations And Risks

Area	Current limitation	Why it matters
Runtime	Generation may take several minutes depending on hardware and document size.	User patience and demo timing can become a factor.
Scale	The pipeline is essentially single-user and local, with no queue or distributed job system.	Not yet ready for many simultaneous users.
Data management	No user accounts or persistent database for transcripts and history.	Generated content is not organized long term.
Format support	The product is centered on PDF input and WAV output.	Other document types and richer output formats are not yet covered.
Model dependency	First run still needs internet to download large models.	True offline use starts after setup, not before.
Robustness	LLM transcript parsing includes repair logic because model output can be imperfect.	Shows the team already encountered and handled real-world model behavior.

13. Future Improvements

- Add authentication and user accounts.
- Store uploaded files, transcripts, and generated audio in a database or object store.
- Support more output modes such as MP3, chaptered audio, or summarized podcasts.
- Allow voice selection, tone control, and transcript editing before synthesis.
- Introduce background jobs, progress percentages, and better concurrency handling.
- Add support for more document types such as DOCX, slides, or web articles.

14. Suggested PPT Structure

Slide	Title	What to include
1	Project title and one-line pitch	Name, tagline, team, and the problem in one sentence.
2	Problem and motivation	Why reading long PDFs is hard and why privacy matters.
3	Solution overview	Simple upload-to-podcast user journey.
4	Architecture	Frontend, FastAPI backend, local models, and output flow.
5	Pipeline	PDF extraction, cleaning, transcript generation, TTS.
6	Tech stack	Key frameworks, models, and deployment tools.
7	Optimizations and uniqueness	Local inference, caching, Apple Silicon support, audio concatenation.
8	UI and demo	Screenshots or flow of the upload page and audio player.

Slide	Title	What to include
9	Challenges and limitations	Runtime, model size, Docker tradeoffs, first-run downloads.
10	Future scope and conclusion	Improvements plus final impact statement.

15. Presenter Talking Points

- Start with the user problem, not the model names. People care first about why the product exists.
- Frame privacy as a major differentiator: no cloud upload of sensitive PDFs after setup.
- Explain that the project is full-stack AI engineering, not just prompting a model.
- When discussing models, keep it simple: one model cleans text, one writes the dialogue, one speaks it.
- If time is short, focus on the flow, architecture, strengths, and demo result.

Suggested closing line: VocalNote shows how local AI can turn passive documents into flexible, private, and engaging learning experiences.

16. Quick Demo Checklist

- Keep a short sample PDF ready before the presentation.
- If live generation is slow, prepare a previously generated WAV for backup playback.
- Show the upload action, the processing state, and the final player plus download button.
- Be ready to explain why the result is a conversation instead of plain narration.
- Mention that the design and deployment choices were made to improve real user experience, not only model output.

17. Final Takeaway

VocalNote is a strong project because it combines a meaningful use case, a privacy-first local AI pipeline, a polished interface, and a clear deployment strategy. It is easy to understand at a high level, but technically rich enough to demonstrate real engineering depth in a presentation.

This brief was prepared from the current repository contents, including the README, FastAPI backend, core podcast pipeline, Docker setup, and React frontend components.